

Jev or Laya? Neither: Why We Fine-Tuned ModernBERT on the Edge for Rural Healthcare AI

An Engineering Postmortem: From Synthetic Data Architecture & Boundary Disambiguation to the 26M Parameter 'Laya Tax' and Sub-150ms Edge Intent Classification

By **Suyash Baoney** • Author of *Beyond AI Guardrails* and *When Bots Pay the Toll* • September 2026

The Core Thesis: The recent debate around 'System 1' decision models (Jev vs. Laya) has exposed an undeniable truth: autoregressive generative LLMs are far too slow, expensive, and non-deterministic for upstream intent routing. But when deploying in low-connectivity rural health networks, neither proprietary cloud APIs (Jev) nor custom-wrapped 421M checkpoints (Laya) provided a clean path. Here is the exact postmortem of our synthetic dataset design, why we shed Laya's 26M deadweight decision heads, how we resolved the Hugging Face TokenizersBackend serialization crash, and how we built a sub-150ms edge router on pure ModernBERT-large (395M).

1. The Backstory: When a Butterfly Evolves into a Dragon

Every production-grade AI system begins as a modest prototype. In our project, **AarogyaMitra**—built to provide conversational clinical triage, government health scheme discovery, and emergency facility routing across 22 Indic languages—the initial vision was a lightweight mobile chatbot assisting rural patients and ASHA community workers. A delicate butterfly.

However, real-world deployment in Tier-2, Tier-3, and remote rural clinics forced that butterfly to grow into a multi-tiered 'dragon': integrating offline GGUF local generation, hybrid spatial engines (Ola Maps + OpenStreetMap Overpass), zero-cost ABDM/104 telemedicine gateways, and real-time field telemetry.

At the frontline of this system sits the **Intent Router**. When a patient frantically inputs: *'My grandfather is having severe crushing chest pain and left arm numbness'*, the system cannot afford to wait 6,000 milliseconds for an autoregressive 7B/70B model to generate a polite JSON response. It requires a sub-150ms, deterministic, non-autoregressive decision to trigger emergency ambulance dispatch immediately.

2. System 1 vs. System 2: Routing Is Not Generation

Inspired by Daniel Kahneman's *Thinking, Fast and Slow*, the AI ecosystem has begun separating **System 1** (fast, non-deliberative pattern recognition) from **System 2** (slow, deliberate reasoning). Autoregressive LLMs predict tokens sequentially ($\mathcal{O}(N)$ forward passes). Using a generative LLM for intent routing burns massive GPU compute and risks catastrophic formatting hallucinations.

Dimension	Autoregressive LLM (System 2)	System 1 Decision Encoders
Mechanism	Sequential token prediction ($\mathcal{O}(N)$ passes)	Single forward pass embedding ($\mathcal{O}(1)$)
Average Latency	3,000 ms – 9,000 ms	30 ms – 150 ms (Pure CPU)
Output Format	Free-form text / JSON string	Calibrated probability vector & logits
Determinism	Probabilistic, subject to temperature/hallucinations	100% deterministic mathematical boundary
Resource Footprint	Requires 8GB–32GB VRAM GPU	Runs on standard 2-core CPU (<300MB RAM)
Edge Viability	Thermal throttling on low-tier hardware	Native ONNX execution on commodity CPUs

3. Jev vs. Laya: The Commercial Landscape vs. Edge Reality

The market recently rallied around two pioneering System 1 implementations: **Jev** (TypeSafe AI) and **Laya** (Convai Innovations), standardizing decisions into three primitives: *Choice* (multi-class), *Score* (ordinal), and *Noul* (Boolean).

- **Why Jev Failed the Spec:** Jev is delivered exclusively as a closed, proprietary SaaS cloud API. In rural healthcare, transmitting sensitive patient narratives over external cloud APIs violates zero-trust data protection principles and completely fails whenever rural 2G connectivity drops.
- **The Laya Checkpoint Reality:** Convai's Laya (421M parameters) is open-weight under Apache 2.0. However, peeling back the checkpoint reveals an essential architectural fact: `convaiinnovations/laya` is not a fundamentally different foundation model from scratch—it is `answerdotai/ModernBERT-large` (395M) with approximately **26M parameters of custom, proprietary decision heads** bolted on (specifically Choice, Score, and Noul heads designed for Convai's reinforcement learning agent API).

4. The 26M Parameter 'Laya Tax' & The Kaggle Postmortem

AarogyaMitra required a standard 6-class sequence classification head (EMERGENCY_CRITICAL, FACILITY_LOCATOR, GOVT_SCHEME_ELIGIBILITY, MEDICINE_GENERIC_SEARCH, SYMPTOM_TRIAGE_REMEDY, OUT_OF_SCOPE_GENERAL). Retaining Laya's 26M custom parameters was deadweight compute overhead. During training and export, two distinct issues occurred:

Issue A: Non-Standard Architecture Crash (During Fine-Tuning Setup)

- **Symptom:** `AutoModelForSequenceClassification.from_pretrained('convaiinnovations/laya')` threw architecture mismatch errors, unexpected keys, and missing classification heads.
- **Cause:** Laya was packaged around Convai's custom agent scripts (`rl_agent_api.py`, nested encoder weights, proprietary option heads) rather than a standard Hugging Face sequence classification layout.
- **Fix:** Bypassed the unstable wrapper entirely, ingested upstream `answerdotai/ModernBERT-large` (395M), and attached a standard linear sequence classification head.

Issue B: The TokenizersBackend Crash (During Kaggle ONNX Export)

- **Symptom:** The ONNX export script failed with: `ValueError: Tokenizer class TokenizersBackend does not exist or is not currently imported`.
- **Cause:** Pushing the fine-tuned model to Hugging Face (`TheUsurper09/aarogyamitra-laya-intent-router`) triggered a known fast tokenizer serializer bug, writing `"tokenizer_class": "TokenizersBackend"` into `tokenizer_config.json`.
- **Fix:** Fine-tuning never alters vocabulary or token IDs. We decoupled tokenizer loading directly from upstream: `AutoTokenizer.from_pretrained('answerdotai/ModernBERT-large')`, cleanly bypassing the corrupted configuration file.

5. Synthetic Data Engineering: The Architecture & Thought Process

To train our router without compromising patient privacy (PHI), we engineered a domain-specific synthetic generator (`generate_laya_dataset.py`). The script is not random Markov babble; it embodies four deliberate architectural principles:

A. Grounded Clinical Seed Anchors: Each of the 6 classes originates from explicit, hand-curated seed templates capturing authentic clinical urgency, Indian geographical contexts (Wardha, Shivpuri, Vidisha, Gwalior), and rural dialect nuances.

B. Combinatorial Slot Permutation Matrices: The generator dynamically cross-multiplies domain vectors:

- *Geographic & Facility Slots:* Tier-2/3 cities $\times$ facility types (PHC, CHC, Jan Aushadhi, Blood Banks, 24/7 ICU, PIN codes 440001–800001).
- *Pharmacology Salt Mappings:* Brand-to-salt pairs (Dolo 650 $\rightarrow$ Paracetamol, Clavam 625 $\rightarrow$ Amoxicillin/Clavulanate, Telma 40 $\rightarrow$ Telmisartan, Glycomet $\rightarrow$ Metformin) and dosage formats (e.g., 'Tab Azithro 500 OD x 3 days').
- *Symptom & Seasonal Vectoring:* ICMR-approved home remedies, mild triage versus red flags, and monsoon outbreaks (Dengue, Chikungunya, Typhoid, Malaria).
- *Out-of-Scope Linguistic Diversity:* Greetings across Indic languages (Namaste, Pranam, Vanakkam, Sat Sri Akal, Khamma Ghani) and out-of-domain rejection guardrails (math, cricket, politics).

C. Boundary Disambiguation (The Hardest Thought Process): Overlapping semantic queries represent the primary point of failure in clinical routing. We embedded deterministic boundary rules directly into the synthetic seed distribution:

- *Spatial vs. Pharmacology:* 'Where can I buy cheap generic Paracetamol near me?' contains drug terms, but physical acquisition intent forces it to FACILITY_LOCATOR.
- *Spatial vs. Policy:* 'List hospitals in Wardha that accept Ayushman Bharat card' contains scheme names, but routing to an empaneled facility forces it to FACILITY_LOCATOR.
- *Scheme Eligibility:* 'Does Ayushman card cover pre-existing heart surgery costs?' is strictly routed to GOVT_SCHEME_ELIGIBILITY for vector RAG retrieval.

D. Strict Stratification & Deduplication: Normalized, deduplicated, and balanced to exactly 200 distinct samples per class (1,200 rows total) to guarantee zero class imbalance during cross-entropy loss computation.

```
# Excerpt: Synthetic Data Generation Engine (generate_laya_dataset.py)
CLASSES = ["EMERGENCY_CRITICAL", "FACILITY_LOCATOR", "GOVT_SCHEME_ELIGIBILITY", "MEDICINE_GENERIC_SEARCH", "SYMPTOM_TRIAGE_REMEDY", "OUT_OF_SCOPE_GENERAL"]
def generate_augmented_class_2():
    # FACILITY_LOCATOR items = list(EXPLICIT_SEEDS["FACILITY_LOCATOR"])
    types = ["Jan Aushadhi Kendra", "Primary Health Center (PHC)", "Community Health Center (CHC)", "District Hospital", "Blood Bank"]
    for city in CITIES:
        for t in types:
            items.append(f"Where is the nearest {t} in {city}?")
            items.append(f"Is there any 24/7 {t} open right now near {city}?")
        for pin in [440001, 462001, 474001, 452001, 482001, 473001]:
            items.append(f"Nearest Jan Aushadhi medical store near PIN code {pin}.")
    return items[:200]
```

6. Taming Catastrophic Forgetting on Complex Queries

A critical failure mode when fine-tuning transformer encoders is **catastrophic forgetting**: when teaching the model to understand complex, compound multi-clause sentences, the classifier degrades on simple, single-intent queries. We solved this with a three-tier framework:

1. **Experience Replay Buffers:** Fixed 40% replay quota of atomic baseline queries alongside 60% compound synthetic sentences during training.
2. **Elastic Weight Regularization (KL Penalty):** Penalizing aggressive weight shifts against the base ModernBERT encoder representations to preserve geometric feature stability.
3. **DPO Alignment Roadmap:** Direct Preference Optimization to reward primary gatekeeper routing on multi-intent compound inputs while extracting secondary parameters into structured slot metadata.

7. ONNX INT8 Quantization & Field Telemetry

The fine-tuned ModernBERT model was compiled to ONNX format with 8-bit dynamic integer quantization (INT8), yielding sub-150ms execution on standard dual-core CPUs.

Pipeline Node	Execution Target	Avg Latency (ms)	P95 Latency (ms)	Operational Status
ModernBERT Intent Classifier	Intel/AMD x86_64 (Pure CPU)	112.4 ms	166.2 ms	Production Validated
CTranslate2 NLLB Translation	CPU Execution Provider	1,240.0 ms	1,795.0 ms	Production Validated
Qdrant Vector Retrieval (RAG)	Local In-Memory Vector Store	185.0 ms	204.0 ms	Production Validated
Local Qwen 3.5 2B (GGUF)	Local Q4_K_M (Pure CPU)	5,620.0 ms	12,434.0 ms	Production Validated

8. Engineering Takeaways

- 1. Don't pay the SaaS tax for classification:** Proprietary black-box APIs like Jev introduce unnecessary failure points, latency overheads, and privacy liabilities in edge healthcare.
- 2. Beware of bolted-on parameter bloat:** Laya's extra 26M parameters were unnecessary for standard multiclass intent routing. Stripping back to base ModernBERT gave us lower latency, smaller binary sizes, and standard Hugging Face compatibility.
- 3. Synthetic data requires boundary discipline:** High-performance intent routing depends on intentional disambiguation seeds (spatial over generic medicine, spatial over scheme) and balanced class stratification.
- 4. ModernBERT is an edge workhorse:** RoPE embeddings, GeLU activations, and INT8 ONNX execution make ModernBERT the gold standard for sub-150ms edge decision engines.

As AarogyaMitra continues its deployment across rural clinics, we will be open-sourcing our synthetic data generation toolchains and benchmark suites. The future of healthcare AI isn't just about bigger models in massive cloud data centers—it's about resilient, instantaneous intelligence operating right at the edge.